The function F_s contains information about the way M would compute on a string s. In particular, $F_s(\text{first})$ is the state that M enters when it is about to move off of the right end of s for the first time, when M is started in its starting state at the left end of s. If M accepts or rejects (either explicitly or by looping) before ever moving off of the right end of s, then $F_s(\text{first}) = \texttt{acc}$ or $\texttt{rej}$ accordingly. Furthermore, for $F_s(q)$ is the state M enters when it is about to move off of the right end of s for the first time, when M is started in state q at the right end of s. As before, if M accepts or rejects before ever moving off of the right end of s, then $F_s(q) = \texttt{acc}$ or $\texttt{rej}$.

Observe that, for any string s and symbol a in Σ, we can determine F_{sa} from a, F_s, and M's transition function. Furthermore, F_ε is predetermined because $F_\varepsilon(\text{first}) = q_0$, the start state of M, and $F_\varepsilon(q) = q$ (because TMs that attempt to move off the leftmost end of the tape just stay in the left-hand cell). Hence we can obtain A's start state and transition function. That completes the first stage of A's design.

To complete the second stage, we assign the accept states of M. For any two strings s and t, if $F_s = F_t$, then M must have the same output on s and t—either both s and t are in $L(M)$ or both are out. Hence A's state at the end of s is enough to determine whether M accepts s. More precisely, we declare a state of A to be accepting if it was assigned to a function F that equals F_s for any string s that M accepts.

3.19 The language A is one of the two languages, $\{0\}$ or $\{1\}$. In either case the language is finite, and hence decidable. If we are not able to determine which of these two languages is A, we won't be able to describe the decider for A, but we can give two TMs, one of which is A's decider.

Chapter 4

4.1
a. Yes
b. No
c. No
d. No
e. No
f. Yes

4.2 Let $EQ_{\text{DFA,REX}} = \{\langle A, R\rangle \mid A \text{ is a DFA}, R \text{ is a regular expression and } L(A) = L(R)\}$. The following TM E decides $EQ_{\text{DFA,REX}}$.

E = "On input $\langle A, R\rangle$:

1. Convert regular expression R to an equivalent DFA B using the procedure given in Theorem 1.28.
2. Use the TM C for deciding EQ_{DFA} given in Theorem 4.5, on input $\langle A, B\rangle$.

3. If R accepts, *accept*. If R rejects, *reject*."

4.3 Let $ALL_{\text{DFA}} = \{\langle A\rangle | \ A \text{ is a DFA that recognizes } \Sigma^*\}$. The following TM L decides ALL_{DFA}.

L = "On input $\langle A\rangle$ where A is a DFA:

1. Construct DFA B that recognizes $\overline{L(A)}$ as described in Exercise 1.10.
2. Run TM T from Theorem 4.4 on input $\langle B\rangle$, where T decides E_{DFA}.
3. If T accepts, *accept*. If T rejects, *reject*."

4.4 Let $A\varepsilon_{\text{CFG}} = \{\langle G\rangle | \ G \text{ is a CFG that generates } \varepsilon\}$. The following TM V decides $A\varepsilon_{\text{CFG}}$.

V = "On input $\langle G\rangle$ where G is a CFG:

1. Run TM S from Theorem 4.6 on input $\langle G, \varepsilon\rangle$, where S is a decider for A_{CFG}.
2. If S accepts, *accept*. If S rejects, *reject*."

4.5 Let $INFINITE_{\text{DFA}} = \{\langle A\rangle | \ L(A) \text{ is an infinite language}\}$. The following TM I decides $INFINITE_{\text{DFA}}$.

I = "On input $\langle A\rangle$ where A is a DFA:

1. Let k be the number of states of A.
2. Construct a DFA D that accepts strings of length $\geq k$.
3. Construct a DFA M such that $L(M) = L(A) \cap L(D)$.
4. Run TM T from Theorem 4.4 on input $\langle M\rangle$, where T decides E_{DFA}.
5. If T accepts, *reject*. If T rejects, *accept*."

If A accepts a string with length at least k, this string can be must cause A to enter the same state at least twice, so it can be pumped up to obtain infinitely many strings that are accepted by A. Conversely, if A accepts infinitely many strings, it must accept strings of arbitrarily long lengths, in particular, a string of length at least k.

4.6

a. g is one-to-one. f is not one-to-one because $f(1) = f(3)$.

b. g is onto. f is not onto because there does not exist $x \in X$ such that $f(x) = 10$.

c. g is a correspondence because g is one-to-one and onto. f is not a correspondence because f is not one-to-one and onto.

4.7 Suppose $\mathcal{B}$ is countable and a correspondence $f: \mathcal{N} \longrightarrow \mathcal{B}$ exists. We construct x in $\mathcal{B}$ that is not paired with anything in $\mathcal{N}$. Let $x = x_1 x_2 \ldots$. Let $x_i = 0$ if $f(i)_i = 1$, and $x_i = 1$ if $f(i)_i = 0$ where $f(i)_i$ is the ith bit of $f(i)$. Therefore, we ensure that x is not $f(i)$ for any i because it differs from $f(i)$ in the ith symbol, and a contradiction occurs.

4.8 We demonstrate a one-to-one $f: T \longrightarrow \mathcal{N}$. Let $f(i,j,k) = 2^i 3^j 5^k$. Function f is one-to-one because if $a \neq b$, $f(a) \neq f(b)$. Therefore, T is countable.

4.9 Let $\sim$ be the binary relation "is the same size". In other words If $A \sim B$, A and B are the same size. We show that $\sim$ is an equivalence relation. First, $\sim$ is reflexive because the identity function $f(x) = x, \forall x \in A$ is a correspondence $f: A \longrightarrow A$. Second, $\sim$ is symmetric because any correspondence has an inverse, which itself is a correspondence. Third, $\sim$ is transitive because if $A \sim B$ via correspondence f, and $B \sim C$ via correspondence g, then $A \sim C$ via correspondence $f \circ g$ (the composition of f and g). Because $\sim$ is reflexive, symmetric, and transitive, $\sim$ is an equivalence relation.

4.10 The following TM X decides A.

X = "On input $\langle M \rangle$ where M is a DFA:

1. Construct a DFA O that accepts any string containing an odd number of 1s.
2. Construct DFA B such that $L(B) = L(M) \cap L(O)$.
3. Run TM T from Theorem 4.4 on input $\langle B \rangle$, where T decides E_{DFA}.
4. If T accepts, *accept*. If T rejects, *reject*."

4.11 We observe that $L(R) \subseteq L(S)$ if and only if $\overline{L(S)} \cap L(R) = \emptyset$. The following TM X decides A.

X = "On input $\langle R, S \rangle$ where R and S are regular expressions:

1. Construct DFA E such that $L(E) = \overline{L(S)} \cap L(R)$.
2. Run TM T from Theorem 4.4 on input $\langle E \rangle$, where T decides E_{DFA}.
3. If T accepts, *accept*. If T rejects, *reject*."

4.12 We have shown in Problem 2.17 that if C is a context-free language and R is a regular language, then $C \cap R$ is context free. Therefore $1^* \cap L(G)$ is context free. The following TM X decides A.

X = "On input $\langle G \rangle$ where G is a CFG:

1. Construct CFG H such that $L(H) = 1^* \cap L(G)$.
2. Run TM R from Theorem 4.7 on input $\langle H \rangle$, where R decides E_{CFG}.

3. If R accepts, *reject*. If T rejects, *accept*."

4.14 The following TM X decides A.

X = "On input $\langle R \rangle$ where R is a regular expression:

1. Construct DFA E that accepts $\Sigma^*111\Sigma^*$.
2. Construct DFA B such that $L(B) = L(R) \cap L(E)$.
3. Run TM T from Theorem 4.4 on input $\langle B \rangle$, where T decides E_{DFA}.
4. If T accepts, *reject*. If T rejects, *accept*."

4.15 Let $A = \{v |\ v \text{ is of the form } ww^{\mathcal{R}} \text{ for } w \in \{0,1\}^*\}$. A is a CFL. Problem 2.17 showed that $B \cap A$ is context free if B is regular. The following TM X decides E.

X = "On input $\langle M \rangle$ where M is a DFA:

1. Let $C = L(M) \cap A$. Let G be the CFG of C.
2. Run TM R from Theorem 4.7 on input $\langle G \rangle$, where R decides E_{CFG}.
3. If R accepts, *reject*. If R rejects, *accept*."

X decides E because a DFA M accepts some string of the form $ww^{\mathcal{R}}$ if and only if $L(M) \cap A \neq \emptyset$.

4.16 For any DFAs A and B, $L(A) = L(B)$ if and only if A and B accept the same strings up to length mn, where m and n are the numbers of states of A and B, respectively. Obviously, if $L(A) = L(B)$, A and B will accept the same strings. If $L(a) \neq L(B)$, we need to show that the languages differ on some string s of length at most mn. Let t be a shortest string on which the languages differ. Let l be the length of t. If $l \leq mn$ we are done. If not, consider the sequence of states $q_0, q_1, \ldots, q_l$ that A enters on input t, and the sequence of states $r_0, r_1, \ldots, r_l$ that B enters on input t. Because A has m states and B has n states, only mn distinct pairs (q, r) exist where q is a state of A and r is a state of B. By the pigeon hole principle, two pairs of states (q_i, r_i) and (q_j, r_j) must be identical. If we remove the portion of t from i to $j-1$ we obtain a shorter string on which A and B behave as they would on t. Hence we have found a shorter string on which the two languages differ, even though t was supposed to be shortest. Hence t must be no longer than mn.

4.17 We need to prove both directions. To handle the easier one first, assume that D exists. A TM recognizing C operates on input x by going through each possible string y and testing whether $\langle x, y \rangle \in D$. If such a y is ever found, *accept*; if not, just continue searching.

For the other direction, assume that C is recognized by TM M. Define a language B to be $\{\langle x, y\rangle |\ M$ accepts x within $|y|$ steps$\}$. Language B is decidable, and if $x \in C$ then M accepts x within some number of steps, so $\langle x, y\rangle \in B$ for any sufficiently long y, but if $x \notin C$ then $\langle x, y\rangle \notin C$ for any y.

4.18 Let A and B be two languages such that $A \cap B = \emptyset$, and $\overline{A}$ and $\overline{B}$ are recognizable. Let J be the TM recognizing $\overline{A}$ and K be the TM recognizing $\overline{B}$. We will show that the language decided by TM T separates A and B.

T = "On input w:

1. Simulate J and K on w by alternating the steps of the two machines.
2. If J accepts first, *reject*. If K accepts first, *accept*."

The algorithm T terminates because $\overline{A} \cup \overline{B} = \Sigma^*$. So either J or K will accept w eventually. $A \subseteq C$ because if $w \in A$, w will not be recognized by J and will be accepted by K first. $B \subseteq \overline{C}$ because if $w \in B$, w will not be recognized by K and will be accepted by J first. Therefore, C separates A and B.

4.19 For any language A, let $A^{\mathcal{R}} = \{w^{\mathcal{R}} |\ w \in A\}$. If $\langle M\rangle \in S$, then $L(M) = L(M)^{\mathcal{R}}$. The following TM T decides S.

T = "On input $\langle M\rangle$, where M is a DFA:

1. Construct DFA N that recognizes $L(M)^{\mathcal{R}}$.
2. Run TM F from Theorem 4.5 on $\langle M, N\rangle$, where F is the Turing machine deciding EQ_{DFA}.
3. If F accepts, *accept*. If F rejects, *reject*."

4.20 Let $U = \{\langle P\rangle |\ P$ is a PDA that has useless states$\}$. The following TM T decides U.

T = "On input $\langle P\rangle$, where U is a PDA:

1. For each state q of P:
2. Modify P so that q is the only accept state.
3. Use the decider for E_{PDA} to test whether the modified PDA's language is empty. If it is, *accept*. If it is not, continue.
4. At this point, all states have been shown to be useful, so *reject*."

4.21 Use the diagonalization method to obtain D. Let $\langle M_1\rangle, \langle M_2\rangle, \ldots$ be the output of an enumerator for A. Let $\Sigma = \{s_1, s_2, \ldots\}$ be a list of all strings over the alphabet. The algorithm for D is:

D = "On input w:

1. Let i be the index of w on the list of all strings, that is, $s_i = w$.
2. Run $\langle M_i\rangle$ on input w.

3. If it accepts, *reject*. If it rejects, *accept*."

D is a decider because each M_i is a decider. But D doesn't appear on the enumeration for A, because it differs from M_i on input s_i.

4.22 Let K be the enumerator for B. The following TM L is the lexicographic-order enumerator for C.

L = "Ignore the input.

1. Enumerate machines M_i by K.
2. For each M_i, insert useless states, symbols or transitions to obtain N_i such that $L(M_i) = L(N_i)$ and $\langle N_i \rangle$ is lexicographically larger than all $\langle N_j \rangle$ for $1 \leq j < i$. Output N_i."

Let $\langle N_1 \rangle, \langle N_2 \rangle \ldots$ be the strings enumerated by L. Since $L(N_i) = L(M_i)$ for all i, every machine described in B has an equivalent one in C and vice versa. C is decidable because it enumerated by L in lexicographic order, by virtue of Problem 3.16.

Chapter 5

5.1 Suppose for a contradiction that EQ_{CFG} were decidable. We construct a decider M for $ALL_{\mathsf{CFG}} = \{\langle G \rangle | \ G \text{ is a CFG and } L(G) = \Sigma^*\}$ as follows:

M = "On input $\langle G \rangle$:

1. Construct a CFG H such that $L(H) = \Sigma^*$.
2. Run the decider for EQ_{CFG} on $\langle G, H \rangle$.
3. If it accepts, *accept*. If it rejects, *reject*."

M decides ALL_{CFG} assuming a decider for EQ_{CFG} exists. Since we know ALL_{CFG} is undecidable, we have a contradiction.

5.2 Here is a Turing Machine M which recognizes the complement of EQ_{CFG}:

M = "On input $\langle G, H \rangle$:

1. Lexicographically generate the strings $x \in \Sigma^*$.
2. For each such string x:
3. Test whether $x \in L(G)$ and whether $x \in L(H)$, using the algorithm for A_{CFG}.
4. If one of the tests accepts and the other rejects, *accept*; otherwise, continue."

5.3 Here is a match: $\left[\frac{\mathtt{ab}}{\mathtt{abab}}\right]\left[\frac{\mathtt{ab}}{\mathtt{abab}}\right]\left[\frac{\mathtt{aba}}{\mathtt{b}}\right]\left[\frac{\mathtt{b}}{\mathtt{a}}\right]\left[\frac{\mathtt{b}}{\mathtt{a}}\right]\left[\frac{\mathtt{aa}}{\mathtt{a}}\right]\left[\frac{\mathtt{aa}}{\mathtt{a}}\right]$.